



Sandya Mannarswamy

Welcome to another instalment of CodeSport, in which we'll continue our discussion on how to write efficient and correct code for multithreaded applications. We will also cover the complex issue of deadlock in multithreaded applications.

*T*hanks to all the readers who sent in their feedback on the problems we discussed in last month's column.

We had given a small code snippet of multithreaded code and asked you to find out the potential bug hiding in it. Congratulations to our readers Siva Kumar, Vivek Goel and Arjun Pakrashi for getting the answer correct. As pointed out by these readers, the code snippet had a potential deadlock situation. Here is the buggy code snippet from the takeaway problem:

```
void BookTicket(int row, int column)
{
    pthread_mutex_lock(&row_lock);
    pthread_mutex_lock(&column_lock);
    ticket[row][column].status = 'booked';
    pthread_mutex_unlock(&column_lock);
    pthread_mutex_unlock(&row_lock);
}

void CancelTicket(int row, int column)
{
    pthread_mutex_lock(&column_lock);
    pthread_mutex_lock(&row_lock);
    ticket[row][column].status = 'cancelled';
    pthread_mutex_unlock(&row_lock);
    pthread_mutex_unlock(&column_lock);
}
```

Note that two threads can concurrently issue booking and cancellation requests to the same ticket (same row and same column), and the system has to work correctly. The problem is that Thread 1 can enter the critical section 'BookTicket' and acquire the 'row_lock' and concurrently Thread 2 can enter the 'CancelTicket' critical section and acquire the 'column_lock'. Now Thread 1 will wait endlessly for 'column_lock' to be released while Thread 2 will wait for the 'row_lock' to be released. Neither of the threads can make any progress at all. How can you fix the code so that this problem does not occur?

In our example, it is quite easy to see that if we change the lock acquire order in 'CancelTicket' to first acquire 'row_lock' and then acquire 'column_lock', we can avoid the deadlock. Here is the corrected version of 'CancelTicket', which does not cause a deadlock with 'BookTicket':

```
void CancelTicket(int row, int column)
{
    pthread_mutex_lock(&row_lock);
    pthread_mutex_lock(&column_lock);
    ticket[row][column].status = 'cancelled';
    pthread_mutex_unlock(&column_lock);
    pthread_mutex_unlock(&row_lock);
}
```